

Machine Learning

Decision Trees- Regression, Pruning



Dr. G. Omprakash

KLEF Hyderabad (Aziz Nagar) – Dept. of ECE

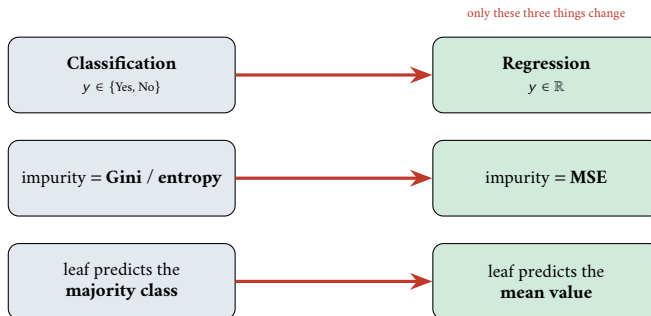
Machine Learning (25SC2107E)

- 1 Decision Trees- Regression
 - Example 1 — One Split
 - Example 2 — Two Splits
 - Problem

- 2 Pruning
 - Pre-Pruning
 - Post-Pruning
 - BIAS-VARIANCE TRADE-OFF

Decision Trees- Regression

From Classification to Regression



The shape of the model

A classification tree outputs *labels* on boxes. A regression tree outputs a **constant number** on each box — so the fitted curve is a **staircase**.

The Splitting Criterion: Mean Squared Error

Impurity of a node

For a node holding values $y_1, \dots, y_n$:

$$\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i, \quad \text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2$$

This is just the **variance** of the node — how *spread out* its values are.

Why the leaf predicts the *mean*

Among all constants c , the value that minimises $\sum_i (y_i - c)^2$ is exactly $c = \bar{y}$.

So once a leaf is fixed, the mean is the best possible answer.

Score of a split

$$\text{MSE}_{\text{split}} = \frac{n_L}{n} \text{MSE}_L + \frac{n_R}{n} \text{MSE}_R$$

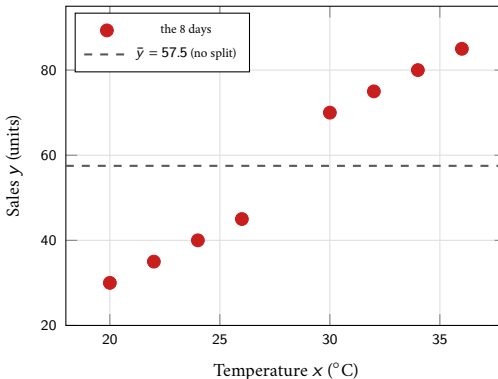
$$\text{Reduction} = \text{MSE}_{\text{parent}} - \text{MSE}_{\text{split}}$$

Pick the cut with the **largest reduction**.

Example 1, The Data

Ice-cream sales on eight days

Temperature x ($^{\circ}\text{C}$)	Sales y (units)
20	30
22	35
24	40
26	45
30	70
32	75
34	80
36	85

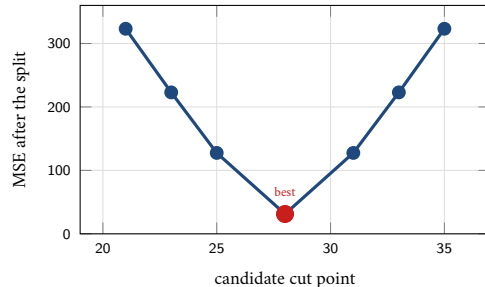


MSE before the split

$$\bar{y} = \frac{460}{8} = 57.5, \quad \text{MSE} = \frac{3450}{8} = 431.25$$

Example 1, Try Every Split, Score It With MSE

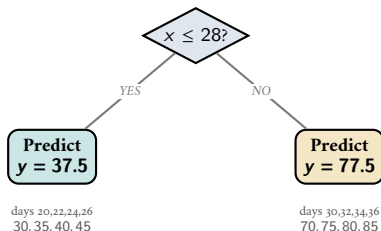
Cut	n_L mean	n_R mean	MSE split	Drop
$x \leq 21$	1 30.0	7 61.4	323.21	108.0
$x \leq 23$	2 32.5	6 65.8	222.92	208.3
$x \leq 25$	3 35.0	5 71.0	127.50	303.8
$x \leq 28$	4 37.5	4 77.5	31.25	400.0
$x \leq 31$	5 44.0	3 80.0	127.50	303.8
$x \leq 33$	6 49.2	2 82.5	222.92	208.3
$x \leq 35$	7 53.6	1 85.0	323.21	108.0



Cut point

Cut points are midpoints of neighbouring x values: 26 and 30 give 28.

Example 1, The Model



The model

$$\hat{f}(x) = \begin{cases} 37.5 & \text{if } x \leq 28 \\ 77.5 & \text{if } x > 28 \end{cases}$$

Predict a new day

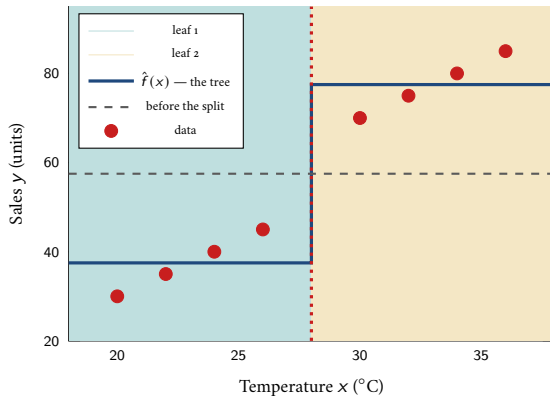
$x = 31^\circ\text{C} \Rightarrow 31 > 28 \Rightarrow \text{predict } \mathbf{77.5}$ units.

Each leaf answers with its own mean

Left leaf: $\frac{30 + 35 + 40 + 45}{4} = 37.5$

Right leaf: $\frac{70 + 75 + 80 + 85}{4} = 77.5$

Example 1, Plot the Model



Observation

- The model is a step, not a line.
- Inside each shaded box the prediction is flat.
- The step sits exactly at the cut, $x = 28$.

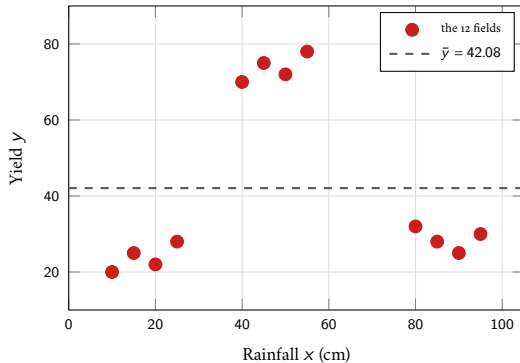
Example 2, The Data

Crop yield against rainfall

Rain	Yield	Rain	Yield
x (cm)	y	x (cm)	y
10	20	50	72
15	25	55	78
20	22	80	32
25	28	85	28
40	70	90	25
45	75	95	30

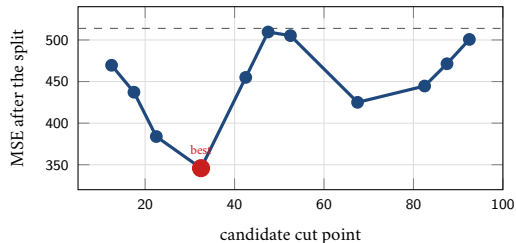
Before any split

$$\bar{y} = 42.08, \quad \text{MSE} = \frac{6166.92}{12} = 513.91$$



Example 2, The First Split

Cut	$n_L n_R$	means L R	MSE split
$x \leq 12.5$	1 11	20.0 44.1	469.58
$x \leq 17.5$	2 10	22.5 46.0	437.21
$x \leq 22.5$	3 9	22.3 48.7	383.89
$x \leq 32.5$	4 8	23.8 51.3	345.85
$x \leq 42.5$	5 7	33.0 48.6	454.98
$x \leq 47.5$	6 6	40.0 44.2	509.57
$x \leq 52.5$	7 5	44.6 38.6	505.24
$x \leq 67.5$	8 4	48.8 28.8	425.02
$x \leq 82.5$	9 3	46.9 27.7	444.63
$x \leq 87.5$	10 2	45.0 27.5	471.38
$x \leq 92.5$	11 1	43.2 30.0	500.64



The winner: Rainfall ≤ 32.5

MSE falls 513.91 $\rightarrow$ 345.85.

Notice the second dip

The curve dips again near $x = 67.5$. Greedy search takes only the deepest dip

Example 2, Recurse Into Both Children

Left child: $x \leq 32.5$

4 fields: 20, 25, 22, 28 mean = 23.75, MSE = 9.19

Cut	MSE split	Drop
$x \leq 12.5$	4.50	4.69
$x \leq 17.5$	7.63	1.56
$x \leq 22.5$	3.17	6.02

Right child: $x > 32.5$

8 fields: 70, 75, 72, 78, 32, 28, 25, 30 mean = 51.25, MSE = 514.19

Cut	MSE split	Drop
$x \leq 42.5$	463.96	50.22
$x \leq 52.5$	247.48	266.70
$x \leq 67.5$	7.94	506.25
$x \leq 82.5$	180.48	333.70
$x \leq 87.5$	326.17	188.02

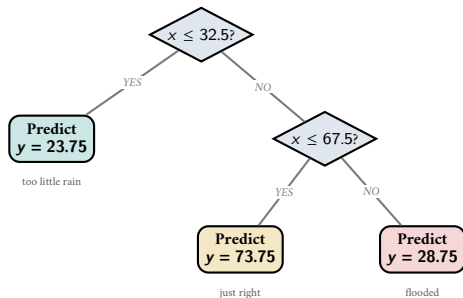
Stopping rule

The best drop here is 6.02, on a node that was already almost flat. A stopping rule (`min_impurity_decrease`) rejects it, and the node becomes a leaf.

Split in the Right child

MSE crashes $514.19 \rightarrow 7.94$. This split separates the **good-rain** fields from the **flooded** ones.

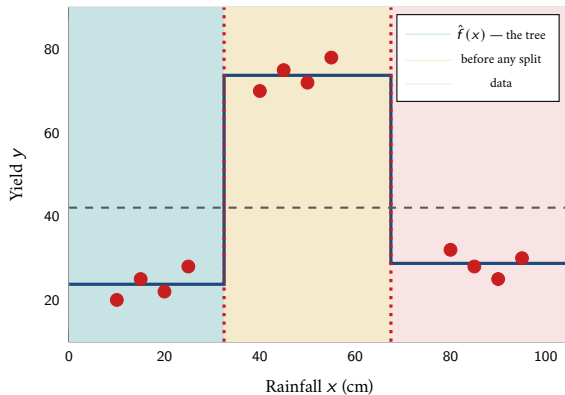
Example 2, How the Model Is Obtained



The model

$$\hat{f}(x) = \begin{cases} 23.75 & x \leq 32.5 \\ 73.75 & 32.5 < x \leq 67.5 \\ 28.75 & x > 67.5 \end{cases}$$

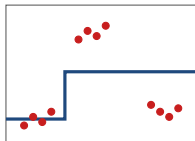
Example 2, Step 5 — Plot the Model



The improvement

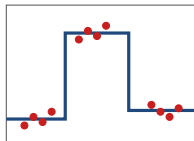
Model	MSE
one constant	42.08
after 1 split	345.85
after 2 splits	8.35

More Steps, Less Error — and Eventually, Memorising



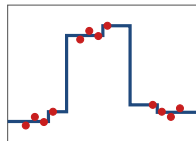
1 split
2 leaves

MSE = 345.85



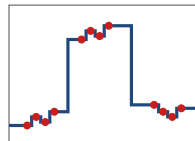
2 splits (our model)
3 leaves

MSE = 8.35



3 levels
7 leaves

MSE = 2.49



no limit at all
12 leaves

MSE = 0.00

The MSE= 0 case

12 leaves for 12 fields. MSE is exactly zero $\implies$ the tree has stored the table, not learned the pattern.

Problem

A food-delivery platform is evaluating regression models to estimate delivery time from tabular data (distance, preparation time, rider rating). The team compares a linear-regression baseline against a top-down greedy regression tree.

Order	Distance (km)	Prep time (min)	Rider rating	Delivery time y (minutes)
O ₁	1.5	8	4.8	18
O ₂	2.5	12	4.5	22
O ₃	3.0	10	4.7	26
O ₄	5.0	15	4.2	38
O ₅	6.5	9	4.6	46

For this mini-dataset of five orders, a candidate split on Distance at 3.5 km divides the data into a Left group (Distance ≤ 3.5) and a Right group (Distance > 3.5).

- 1 Compute the Mean Squared Error of the parent dataset before splitting.
- 2 Compute the predicted leaf output values $\hat{c}_L$ and $\hat{c}_R$ for each group after the split.
- 3 Calculate the weighted post-split MSE and the total reduction in MSE achieved by this split.

Solution — Part 1: MSE Before the Split

The parent node holds all five orders. A regression tree predicts one constant for the whole node, and the constant that minimises squared error is the mean.

$$\bar{y} = \frac{18 + 22 + 26 + 38 + 46}{5} = \frac{150}{5} = 30 \text{ minutes}$$

Order	y_i	$y_i - \bar{y}$	$(y_i - \bar{y})^2$
O ₁	18	-12	144
O ₂	22	-8	64
O ₃	26	-4	16
O ₄	38	+8	64
O ₅	46	+16	256
SSE			544

$$\text{MSE}_{\text{parent}} = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2 = \frac{544}{5} = 108.8$$

Solution — Part 2: The Leaf Output Values

The split at Distance = 3.5 km sends O₁, O₂, O₃ to the left and O₄, O₅ to the right. Each leaf again predicts the mean of the orders inside it.

Left leaf — Distance ≤ 3.5

Order	y_i	$y_i - \hat{c}_L$	$(y_i - \hat{c}_L)^2$
O ₁	18	-4	16
O ₂	22	0	0
O ₃	26	+4	16
SSE _L			32

$$\hat{c}_L = \frac{18 + 22 + 26}{3} = \frac{66}{3} = 22$$

$$\text{MSE}_L = \frac{32}{3} = 10.67$$

Right leaf — Distance > 3.5

Order	y_i	$y_i - \hat{c}_R$	$(y_i - \hat{c}_R)^2$
O ₄	38	-4	16
O ₅	46	+4	16
SSE _R			32

$$\hat{c}_R = \frac{38 + 46}{2} = \frac{84}{2} = 42$$

$$\text{MSE}_R = \frac{32}{2} = 16$$

Solution — Part 3: Weighted MSE and the Reduction

Each child is weighted by the fraction of orders that reach it:

$$\text{MSE}_{\text{split}} = \frac{n_L}{n} \text{MSE}_L + \frac{n_R}{n} \text{MSE}_R = \frac{3}{5} \left(\frac{32}{3} \right) + \frac{2}{5} \left(\frac{32}{2} \right) = 6.4 + 6.4 = 12.8$$

$$\Delta \text{MSE} = \text{MSE}_{\text{parent}} - \text{MSE}_{\text{split}} = 108.8 - 12.8 = 96.0$$

Quantity	Value
MSE before the split	108.8
MSE after the split	12.8
Reduction	96.0 (88.2% of the parent MSE)

Summary — Regression Trees

	Classification	Regression
Target	class	number
Impurity	Gini / entropy	MSE
Leaf says	majority	mean
Best case	$G = 0$	$\text{MSE} = 0$
Worst case	$G = 0.5$	unbounded
Model shape	regions	staircase
sklearn	DecisionTree Classifier	DecisionTree Regressor

The two rules to remember

- **Where to cut:** halfway between neighbouring x values — the average of the last x on the left and the first x on the right.
- **What to predict:** the average of the y values in that leaf.

What happens internally

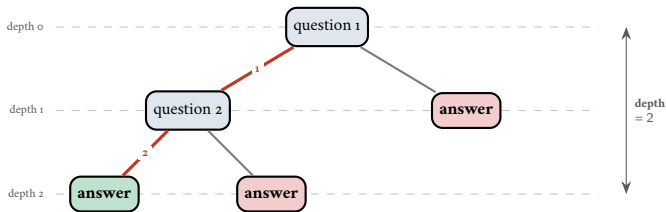
- At a node, compute $\text{MSE} = \frac{1}{n} \sum (y_i - \bar{y})^2$.
- Score every candidate cut by $\frac{n_L}{n} \text{MSE}_L + \frac{n_R}{n} \text{MSE}_R$.
- Keep the cut with the biggest reduction, then recurse.

Two warnings

- A regression tree **cannot extrapolate** — outside the training range it just repeats the end step.
- Squared error is **sensitive to outliers**;
`criterion="absolute_error"` is the robust alternative.

Pruning

What Is the *Depth* of a Tree?



The red path is the longest one. It crosses **2 edges**, so the depth is 2.

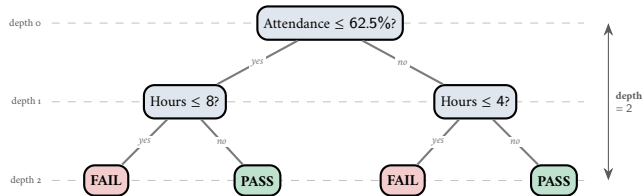
Definition

The number of edges on the **longest** path from the root down to a leaf

Counting

- Count **edges**, not nodes. Root sits at depth **0**.
- A single leaf (no question) has depth **0**.
- `get_depth()` returns depth of a tree.

Balanced Tree



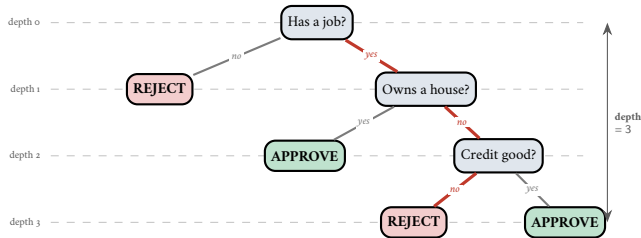
Path from root to leaf	Edges
yes → yes → FAIL	2
yes → no → PASS	2
no → yes → FAIL	2
no → no → PASS	2
longest	2

A depth-2 tree could hold $2^{\text{depth}} = 2^2 = 4$ leaves;

A perfectly balanced tree

Every path is the same length, so every student is asked exactly 2 questions.

Unbalanced Tree



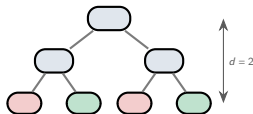
Path from root to leaf	Edges
no → REJECT	1
yes → yes → APPROVE	2
yes → no → yes → APPROVE	3
yes → no → no → REJECT	3
longest	3

The numbers

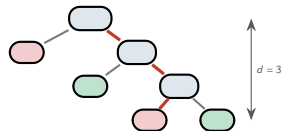
depth = 3 leaves = 4 questions = 3

A depth-3 tree could hold $2^3 = 8$ leaves; this one has 4.

Same Number of Leaves — Different Depth



Example 1: balanced
4 leaves, depth 2



Example 2: lopsided
4 leaves, depth 3

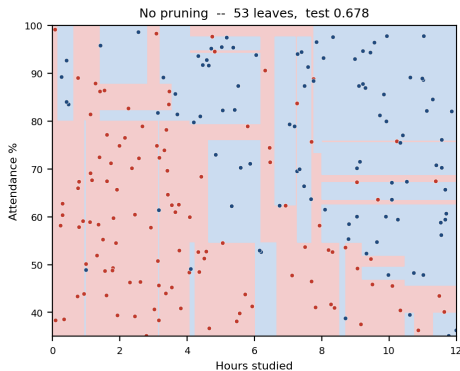
	Ex1	Ex2
Leaves	4	4
Questions	3	3
Shortest path	2	1
Longest path	2	3
Depth	2	3

Depth is not the same as size

Both trees have **4 leaves and 3 questions**. One has depth 2, the other depth 3

The Data

Consider the 300 students data.



- Two features :Hours studied and Attendance.
- 210 students for training, 90 held back for testing.
- Observation: a dot on *matching* shading was classified correctly; a dot on the *opposite* colour is a mistake.
- Count the mistakes here: **zero** — all 210 dots sit on their own colour. That is what forced the tree to carve thin slivers around every stray dot.

Figure: Dots = the 210 training students (dark red = failed, dark blue = passed). Shading = the class the unpruned tree predicts at that point (pink = Fail, light blue = Pass.)

The Tree With No Limits

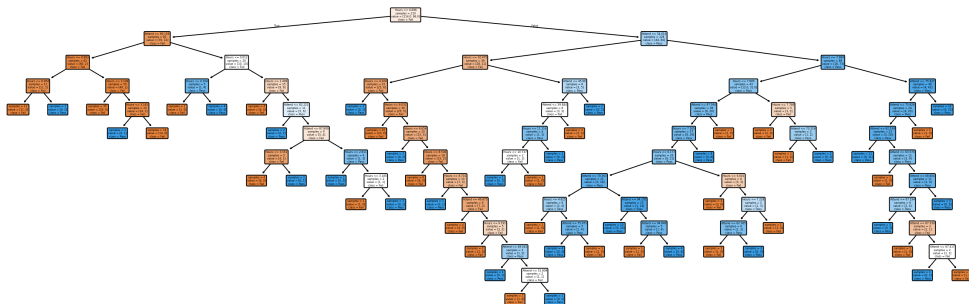


Figure: The real tree, drawn by plot_tree: **depth 12, 53 leaves**

- You won't be able to read it.
- Training accuracy **1.000**, test accuracy **0.678**.
- Many leaves hold one or two students. The tree has stored the register, not the pattern.

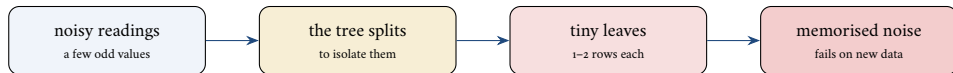
The Code for Tree with No Limits

```

1 import numpy as np
2 from sklearn.tree import DecisionTreeClassifier, plot_tree
3 from sklearn.model_selection import train_test_split
4
5 # ---- make the data -----
6 rng = np.random.default_rng(1)
7 n = 300
8 hours = rng.uniform(0, 12, n)      # study hours per week
9 attend = rng.uniform(35, 100, n)   # attendance %
10 z = 0.45*hours + 0.08*attend - 8.0
11 y = (rng.random(n) < 1/(1 + np.exp(-z))).astype(int)  # 1 = pass
12 X = np.c_[hours, attend]
13
14 Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0)
15
16 # ---- grow a tree with NO limits -----
17 tree = DecisionTreeClassifier(random_state=0).fit(Xtr, ytr)
18
19 print(tree.get_depth(), tree.get_n_leaves())    # 12  53
20 print(tree.score(Xtr, ytr))                    # 1.000  <- perfect
21 print(tree.score(Xte, yte))                    # 0.678  <- poor
22
23 plot_tree(tree, feature_names=["Hours", "Attend"],
24           class_names=["Fail", "Pass"], filled=True)

```

Noisy Data Makes a Tree Grow Too Far



- A faulty measurement looks exactly like a real pattern to the algorithm.
- The greedy search will split the data to reduce training impurity.
- When the classes are imbalanced, this gets worse
- Two remedies: Pre-Pruning stop early and Post-Pruning cut back afterwards .

Pre-Pruning: the Stopping Criteria

Each rule is checked before a split is made. If any rule breaks, the node becomes a leaf.

Criterion	The node becomes a leaf when	scikit-learn
Maximum depth	it already sits d questions below the root	<code>max_depth</code>
Minimum samples to split	it holds fewer than k rows	<code>min_samples_split</code>
Minimum samples per leaf	the split would leave a child with $< k$ rows	<code>min_samples_leaf</code>
Minimum impurity decrease	the best gain available is below a threshold	<code>min_impurity_decrease</code>
Maximum leaf nodes	the tree already has the allowed number of leaves	<code>max_leaf_nodes</code>
Node is pure	every row in it has the same label	(automatic)

- **Depth** limits the model complexity. A depth- d tree has at most 2^d leaves.
- **Minimum leaf size** a leaf holding one odd reading is never created in the first place (Effective on noise).
- **Minimum gain** refuses splits that are not worth a question.

Challenges with Noisy, Imbalanced Data

- **Pre-pruning is short-sighted.**

A split can look worthless now and still be the doorway to two excellent splits beneath it. The rule fires, the door closes, and the tree never finds out.

- **min_samples_leaf can erase the rare class.**

If only 2% of rows are failures, setting min_samples_leaf=50 may make it impossible for any leaf to be mostly failures. The tree then predicts “normal” everywhere and reports 98% accuracy.

Problem	What to do
Rare class ignored	<code>class_weight="balanced"</code>
Accuracy is misleading	score with recall, F1 or AUC, not accuracy
Sensors sometimes fail	keep a <code>MissingIndicator</code> column
Do not know the right limits	tune them by cross-validation

On imbalanced data, always check the validation metric before trusting any stopping rule.

Pre-Pruning: Never Let It Grow

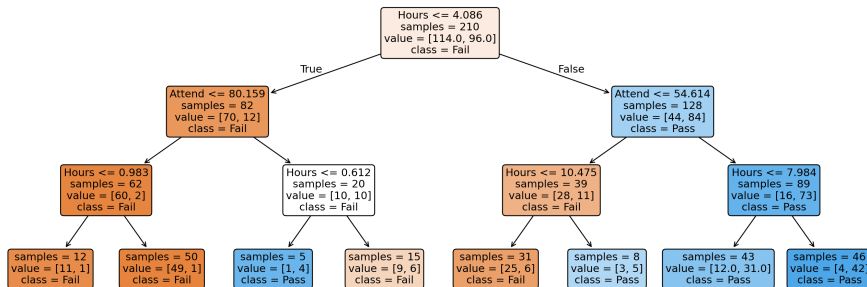


Figure: Growth stopped at max_depth=3: a full, balanced tree of 8 leaves

- Nothing below level 3 was ever built
- Training 0.838, test 0.800 — almost as good as post-pruning (far cheaper to obtain).
- Notice the shape: pre-pruning gives a neat balanced tree, post-pruning gives a lopsided one.

The Code for Pre-Pruning

```

1 # Every limit below is checked BEFORE a split is made.
2
3 early = DecisionTreeClassifier(
4     random_state=0,
5     max_depth=3,          # at most 3 questions
6     min_samples_split=20, # don't split small nodes
7     min_samples_leaf=10,  # keep leaves populated
8     min_impurity_decrease=0.001)
9 early.fit(Xtr, ytr)
10
11 print(early.get_n_leaves())    # 8 (was 53)
12 print(early.score(Xtr, ytr))  # 0.838
13 print(early.score(Xte, yte))  # 0.800
14
15 # tune the limits, again by cross-validation
16 grid = {"max_depth": [2, 3, 4, 5, 6, None],
17         "min_samples_leaf": [1, 5, 10, 20]}
18 gs = GridSearchCV(DecisionTreeClassifier(random_state=0),
19                  grid, cv=5)
20 gs.fit(Xtr, ytr)
21 print(gs.best_params_)
  
```

max_depth	leaves	train	test
1	2	0.733	0.756
2	4	0.814	0.800
3	8	0.838	0.800
4	16	0.862	0.756
5	23	0.895	0.767
none	53	1.000	0.678

- These limits act during the build.
- The deep nodes are never created, so pre-pruning is also the faster option.
- The risk: a split that looks useless now may have led somewhere good.

Post-Pruning: Turning the Knob (α)

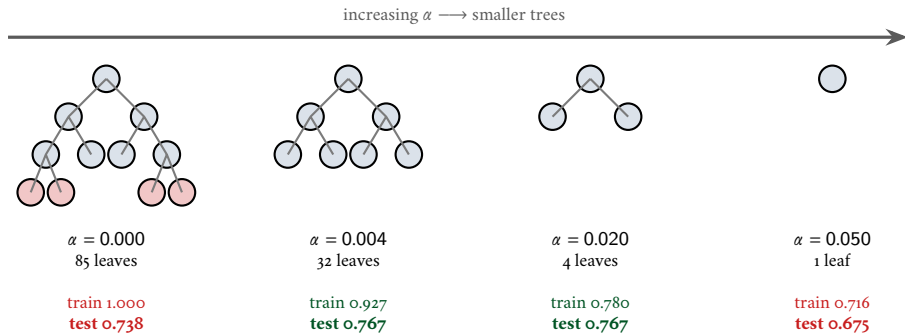
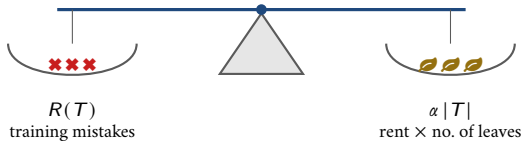


Figure: Numbers are for demonstration purpose

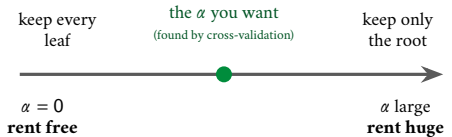
Cost-Complexity Pruning: Charge Rent on Every Leaf

$$\text{minimise } R_\alpha(T) = R(T) + \alpha |T|$$



The rule

A leaf stays only if the mistakes it prevents are worth **more than α** . Otherwise it cannot pay its rent and is cut.



In scikit-learn

`DecisionTreeClassifier(ccp_alpha=...)`; the candidate values come from `cost_complexity_pruning_path()`.

Post-Pruning: Grow It All, Then Cut Back

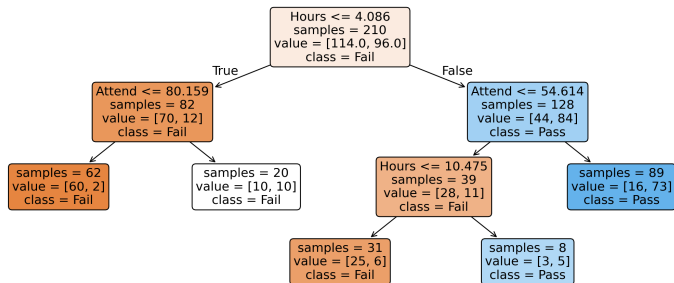


Figure: The same tree after cost-complexity pruning: **53 leaves become 5**

- The tree was grown fully first, then whole branches were collapsed back into leaves.
- Training accuracy falls $1.000 \rightarrow 0.824$. Test accuracy rises $0.678 \rightarrow 0.811$.
- We gave up training accuracy and bought test accuracy. That is the trade pruning always makes.

The Code for Post-Pruning

```

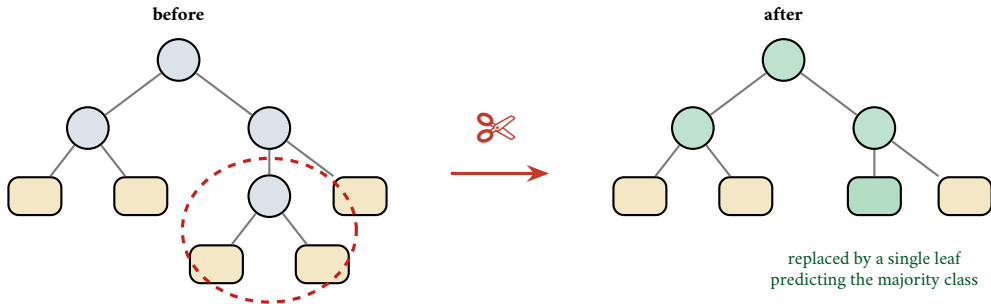
1 # alpha = the "rent" charged for every leaf.
2 # Bigger alpha -> smaller tree.
3
4 pruned = DecisionTreeClassifier(
5     random_state=0,
6     ccp_alpha=0.008)
7 pruned.fit(Xtr, ytr)
8
9 print(pruned.get_n_leaves())    # 5 (was 53)
10 print(pruned.score(Xtr, ytr))  # 0.824
11 print(pruned.score(Xte, yte))  # 0.811
12
13 # where do the candidate alphas come from?
14 path = tree.cost_complexity_pruning_path(Xtr, ytr)
15
16 # choose alpha honestly, by cross-validation
17 from sklearn.model_selection import GridSearchCV
18 gs = GridSearchCV(DecisionTreeClassifier(random_state=0),
19     {"ccp_alpha": path.ccp_alphas}, cv=5)
20 gs.fit(Xtr, ytr)
21 print(gs.best_params_)

```

ccp_alpha	leaves	train	test
0.000	53	1.000	0.678
0.004	45	0.981	0.678
0.006	20	0.905	0.722
0.008	5	0.824	0.811
0.012	4	0.814	0.800
0.040	3	0.814	0.800

- ccp_alpha acts **after** the tree is built.
- Raise it and whole branches collapse into leaves.
- One knob, and the trees it produces are **nested**.

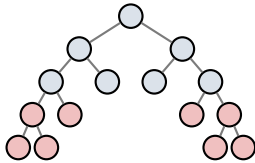
Post-Pruning: Replacing a Subtree with a Leaf



Replacement

Take an internal node, throw away everything below it, and turn it into a leaf whose prediction is the majority class of the training rows that reach it.

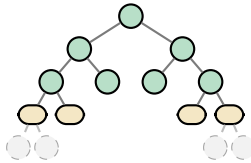
The Three Trees, Side by Side



No pruning

`DecisionTreeClassifier()`

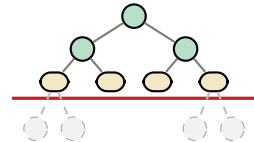
85 leaves • test **0.738**



Post-pruning

`ccp_alpha=0.004`

32 leaves • test **0.767**



Pre-pruning

`max_depth=3`

8 leaves • test **0.754**

Grown freely

A leaf for almost every row. Train 100%, test 73.8%.

Cut afterwards

Sees the whole tree before deciding. Usually the **best accuracy**.

Stopped early

Never builds the big tree. **Fastest**, and easiest to read.

The Three Models, Side by Side

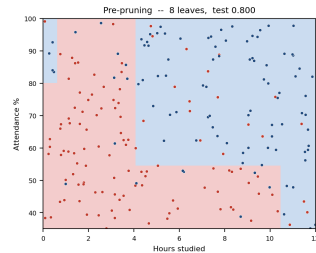
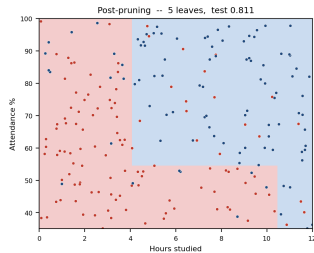
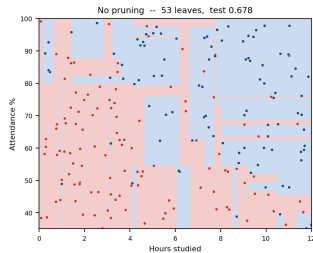


Figure: The same 210 students, three different trees

Model	Setting	Leaves	Train	Test
No pruning	—	53	1.000	0.678
Post-pruning	ccp_alpha=0.008	5	0.824	0.811
Pre-pruning	max_depth=3	8	0.838	0.800

The pruned models are **worse on the training set** and **better on new students**.

Post-Pruning: the Cost-Complexity Measure

Grow the tree fully, then charge *rent* for every leaf and see which tree is cheapest.

$$R_{\alpha}(T) = \underbrace{R(T)}_{\text{error}} + \alpha \underbrace{|T|}_{\text{number of leaves}}$$

- $R(T)$ is the error of tree T , measured on a validation set (or by cross-validation).
- $|T|$ counts the terminal leaves — the size of the model.
- $\alpha \geq 0$ is the complexity parameter: the price of one extra leaf.
- A big tree has small $R(T)$ but large $\alpha|T|$. A small tree has the opposite problem. The winner is whichever total is lowest.

α	Effect
$\alpha = 0$	leaves are free $\Rightarrow$ keep the fully grown tree
α small	prune only the branches that earn almost nothing
α large	leaves are expensive $\Rightarrow$ collapse towards a stump

Choosing Between Two Trees

Given two candidate trees, we compare their R_α values.

- Tree T_A is preferred to T_B exactly when $R_\alpha(T_A) < R_\alpha(T_B)$.
- The two are *equally good* at one particular α , found by setting them equal:

$$R(T_A) + \alpha|T_A| = R(T_B) + \alpha|T_B| \quad \implies \quad \alpha^\star = \frac{R(T_B) - R(T_A)}{|T_A| - |T_B|}$$

- Read $\alpha^\star$ as: *the error the bigger tree saves, per extra leaf it spends*.
- If the real α is below $\alpha^\star$, the extra leaves are cheap — keep the bigger tree.
- If α is above $\alpha^\star$, the leaves cost more than they save — prune.
- Sweeping α from 0 upwards therefore produces a nested sequence of trees, each contained in the previous one. Cross-validation picks one of them.

Problem — Pruning a Fraud-Detection Tree

A bank monitors *transaction amount*, *location mismatch* and *time of day* to predict card fraud. The data is noisy, highly imbalanced (fraud is rare), and some fields are occasionally missing.

- A. Outline three pre-pruning stopping criteria used during top-down greedy construction of a decision tree, to avoid over-growing nodes on noisy transaction data.
- B. A fully grown tree T_0 has validation error $R(T_0) = 0.10$ with $|T_0| = 20$ leaves. Two pruned candidates are available:

Tree	Validation error $R(T)$	Leaves $ T $
T_0 (fully grown)	0.10	20
T_1	0.16	8
T_2	0.22	4

Calculate $R_\alpha(T) = R(T) + \alpha|T|$ for all three trees when $\alpha = 0.008$, and determine which tree is selected.

- C. Find the value of α at which T_0 and T_1 are equally good, and state which tree is preferred on either side of it.

Solution — Part A: Three Stopping Criteria

- 1 **Maximum depth** (`max_depth`).

Stop splitting once a node lies d questions below the root. This caps the model at 2^d leaves and prevents long chains of splits that exist only to chase a few odd sensor readings.

- 2 **Minimum samples per leaf** (`min_samples_leaf`).

Reject any split that would leave a child with fewer than k transactions. A leaf built around one malfunctioning reading is therefore never created. This is the most direct defence against noise.

- 3 **Minimum impurity decrease** (`min_impurity_decrease`).

Make the split only if the weighted impurity drop exceeds a threshold. Splits that barely help are refused, so the tree does not spend questions on noise.

- *Also acceptable:* `min_samples_split` (do not split a node that is already small), or `max_leaf_nodes` (cap the total number of leaves).

Solution — Part B: Cost-Complexity at $\alpha = 0.008$

Apply $R_\alpha(T) = R(T) + \alpha|T|$ to each tree in turn.

$$R_\alpha(T_0) = 0.10 + 0.008 \times 20 = 0.10 + 0.160 = \mathbf{0.260}$$

$$R_\alpha(T_1) = 0.16 + 0.008 \times 8 = 0.16 + 0.064 = \mathbf{0.224}$$

$$R_\alpha(T_2) = 0.22 + 0.008 \times 4 = 0.22 + 0.032 = \mathbf{0.252}$$

Tree	$R(T)$	$ T $	$\alpha T $	$R_\alpha(T)$	Rank
T_0 (fully grown)	0.10	20	0.160	0.260	3rd
T_1	0.16	8	0.064	0.224	1st
T_2	0.22	4	0.032	0.252	2nd

- T_1 is selected, with the lowest cost-complexity measure 0.224.

Solution — Part C: The Break-Even α

Set the two cost-complexity measures equal and solve for α :

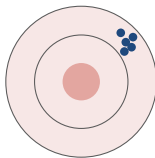
$$0.10 + 20\alpha = 0.16 + 8\alpha \implies 12\alpha = 0.06 \implies \alpha^* = \frac{0.06}{12} = \mathbf{0.005}$$

α	$R_\alpha(T_0)$	$R_\alpha(T_1)$	$R_\alpha(T_2)$	Selected
0.000	0.100	0.160	0.220	T_0 — leaves are free
0.002	0.140	0.176	0.228	T_0
0.005	0.200	0.200	0.240	T_0 and T_1 tie
0.008	0.260	0.224	0.252	T_1 — the question's case
0.015	0.400	0.280	0.280	T_1 and T_2 tie
0.030	0.700	0.400	0.340	T_2 — leaves are expensive

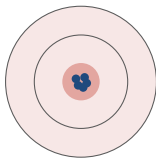
- **Answer:** $\alpha^* = 0.005$. For $\alpha < 0.005$ keep T_0 ; for $\alpha > 0.005$ prefer T_1 . Since the question uses $\alpha = 0.008 > 0.005$, pruning to T_1 is correct — as Part B found.

Bias and Variance in a Tree

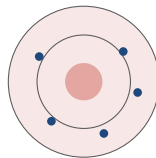
Depth is the one dial that controls how complicated a tree may become. Turning it changes **which kind of mistake** the tree makes.



HIGH bias, low variance
a shallow tree



low bias, low variance
the right depth



low bias, HIGH variance
a deep tree

Figure: Bull's-eye = the truth. Each dot = the tree you get from **one** random sample.

- **Bias** — how far the *average* tree sits from the truth. Caused by a model too simple to represent it.
- **Variance** — how much the tree *changes* when you redraw the data. Caused by a model flexible enough to chase noise.
- For a fixed amount of data, pushing one down usually pushes the other up. That is the **trade-off**.

The Expected test error

For squared-error loss, the expected test error splits into exactly three pieces:

$$\underbrace{\mathbb{E}[(y - \hat{f}(x))^2]}_{\text{expected test error}} = \underbrace{(\mathbb{E}[\hat{f}(x)] - f(x))^2}_{\text{Bias}^2} + \underbrace{\text{Var}(\hat{f}(x))}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{noise}}$$

- $f(x)$ is the truth; $\hat{f}(x)$ is the tree we happened to fit; the expectation is over *re-drawing the training data*.
- σ^2 is the irreducible noise. No model of any depth can beat it.
- So the only thing we control is $\text{Bias}^2 + \text{Variance}$ — and depth moves both.

Experiment

Consider the following experiment to observe Bias-Variance trade-off

The experiment

Truth	a smooth S-shaped curve: exam score against hours studied
Noise	$\sigma = 5$, so the noise floor is $\sigma^2 = 25$
Each sample	120 students, drawn afresh
Repeats	300 times, at every depth

Eight Trees, Three Depths

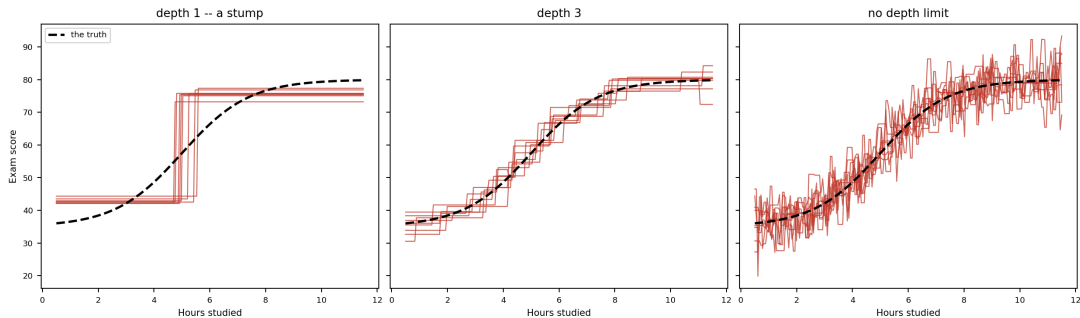
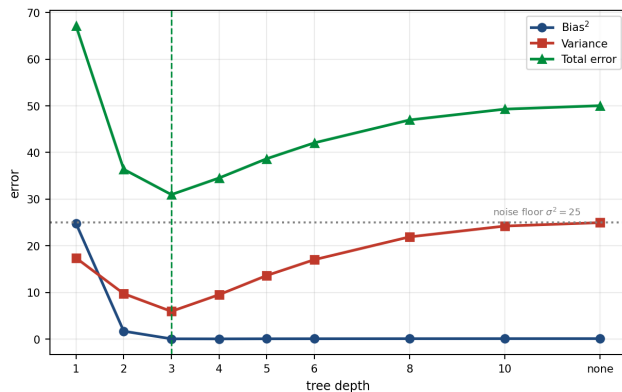


Figure: Each red line is a tree fitted to a **different** sample of 120 students. Dashed black is the truth.

- **Left (depth 1).** The eight fits lie almost on top of each other — very low variance — but all of them miss the curve badly. That gap is bias.
- **Middle (depth 3).** The fits agree with each other *and* follow the truth.
- **Right (no limit).** Wild scribbles that disagree everywhere — huge variance — though on average they trace the right shape.

The Measured Trade-Off

Depth	Leaves	Bias ²	Var	Total
1	2	24.74	17.37	67.11
2	4	1.68	9.72	36.40
3	8	0.04	5.93	30.97
4	16	0.03	9.50	34.53
5	29	0.05	13.61	38.66
6	46	0.06	17.01	42.07
8	82	0.07	21.90	46.97
10	105	0.08	24.22	49.30
none	120	0.08	24.96	50.04



- Bias² collapses fast: 24.74 $\rightarrow$ 0.04 by depth 3, then stays flat.
- Variance climbs steadily: 5.93 $\rightarrow$ 24.96, and never stops rising.
- Total error is U-shaped, lowest at depth 3.

Shallow Vs Deep

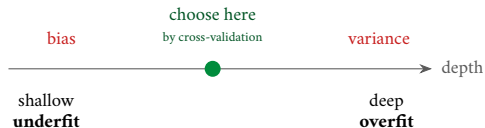
	Shallow tree (depth 1-2)	Deep tree (no limit)
Bias	high – too rigid to fit	very low – fits anything
Variance	low – stable across samples	high – redrawn every time
Training error	high	zero
Test error	high	high (for the other reason)
Called	<i>underfitting</i>	<i>overfitting</i>
Leaves hold	many students	one or two students
More data helps?	no	yes
Our numbers	$B^2 = 24.74, V = 17.37$	$B^2 = 0.08, V = 24.96$

- More data cures variance, not bias.

Controlling the Trade-Off

If the tree is too shallow (high bias):

- Allow more depth.
- Lower `min_samples_leaf`.
- Add better features.



If the tree is too deep (high variance):

- Cap `max_depth`, or post-prune with `ccp_alpha`.
- Raise `min_samples_leaf`.
- Collect more data.
- Average many trees — a random forest.

The special escape route

Pruning fights variance by giving back some bias.

A **random forest** does better: it averages many deep trees, so the variance falls without the bias rising. That is why forests beat a single pruned tree.

Summary — Bias and Variance in Decision Trees

- Expected test error = $\text{Bias}^2 + \text{Variance} + \sigma^2$. Only the first two are ours to control.
- **Shallow trees have high bias.** A depth- d tree has at most 2^d leaves, so it has at most 2^d answers available. Our stump: $\text{Bias}^2 = 24.74$, and no amount of extra data would help.
- **Deep trees have high variance.** Each leaf averages m students, so its variance is about σ^2/m . Deeper means smaller m . At the full tree $m = 1$ and the variance reached 24.96 — essentially the noise itself.
- **Total error is U-shaped in depth.** Here the best depth was 3, with total error 30.97 against 67.11 for the stump and 50.04 for the unpruned tree.
- **Diagnose with the training score:** high training error $\Rightarrow$ bias; near-zero training error but poor test error $\Rightarrow$ variance.
- **Fix bias** with more depth or better features; **fix variance** with pruning, more data, or by averaging many trees in a forest.

Trees: the Good, the Bad

Advantages

- You can read them. A doctor or a teacher can check the rule by eye.
- No preprocessing. No scaling, no standardising.
- Handles numbers, counts and categories together.
- Missing values and outliers barely hurt.
- Prediction is fast: a few comparisons.

Disadvantages

- Unstable. A few different students $\Rightarrow$ a different tree.
- Overfits greedily unless pruned.
- Only axis-parallel cuts — a diagonal boundary needs a staircase of them.
- Cannot extrapolate beyond the training range.
- Greedy, so never guaranteed to be the best tree.

Summary — Decision Trees

What a tree is

- A chain of yes/no questions, learned from data.
- Equivalently: the feature space chopped into boxes, one answer per box.
- Prediction = one walk from root to leaf.

How it is grown

- Greedy: best split *now*, never look back.
- Score = drop in impurity, weighted by group size.
- Gini $1 - \sum p_c^2$; Entropy $-\sum p_c \log_2 p_c$; MSE for numbers.

How it is controlled

- Free growth $\Rightarrow$ 100% train, 73.8% test. Memorising.
- Pre-prune: `max_depth`, `min_samples_leaf`.
- Post-prune: `ccp_alpha`, rent per leaf.
- Depth 5 gave the best test score, 76.7%.

The one line to remember

A single deep tree has low bias and high variance.